- **Accuracy.** This is defined as the ratio of the number of correct predictions to the total number of predictions made. We report top-1 classification accuracy on the test(val) sets of ID datasets.
- **FPR (95% TPR).** This metric stands for 'False Positive Rate at 95% True Positive Rate'. It measures the proportion of negative instances that are incorrectly classified as positive when the true positive rate is 95%. A lower FPR at 95% TPR is desirable as it indicates fewer false alarms while maintaining a high rate of correctly identified true positives.
- **Detection Error (95% TPR).** Detection Error at 95% TPR is a metric that quantifies the overall error rate when the model achieves a true positive rate of 95%. It combines false negatives and false positives to provide a single measure of error. Lower detection error values indicate better performance, as the model successfully identifies more true positives with fewer errors.
- **AUROC** is short for Area Under the Receiver Operating Characteristic Curve(AUROC). This metric measures the ability of a model to distinguish between in-distribution and OOD samples. The ROC curve plots the true positive rate against the false positive rate at various threshold settings. The AUROC is the area under this curve, with higher values (closer to 1.0) indicating better discrimination between in-distribution and OOD samples.
- **AURP** is short for Area Under the Precision-Recall Curve (AUPR), this metric is particularly useful in scenarios where there is a class imbalance (a significant difference in the number of in-distribution and OOD samples). It plots precision (the proportion of true positives among positive predictions) against recall (the proportion of true positives identified). Higher AUPR values suggest better model performance, especially in terms of handling the balance between precision and recall.

Implementation Details. Our Split Ensemble model was trained over 200 epochs using a single NVIDIA A100 GPU with 80GB of memory, for experiments involving CIFAR-10, CIFAR-100, and Tiny ImageNet datasets. For the larger-scale ImageNet dataset, we employ 8 NVIDIA A100 GPUs, each with 80GB memory, to handle the increased computational demands. We use an SGD optimizer with a momentum of 0.9 and weight decay of 0.0005. We also adopt a 200-epoch cosine learning rate schedule with 10 warm-up epochs and a batchsize of 256. Our experiments typically run for approximately 2 hours on both CIFAR-10 and CIFAR-100 datasets, whereas on the Tiny ImageNet and ImageNet datasets, they take approximately 10 hours and 24 hours, respectively. We employ data augmentation techniques such as rotation and flip during the training phase, while the testing phase does not involve data augmentation. As for the backbone models in our experiments, we utilize the standard ResNet-18 and ResNet-34 architectures. We heuristically decide the number of submodels in the Split-Ensemble via ablation study, where we find 8 submodels for ImageNet-1K and 5 submodels for other datasets leads to the best performance in both ID and OOD detection. The classes are grouped based on semantic similarity into subtasks for the submodels to learn. For OOD detection score computation, we use the max softmax probability for the single model, max softmax probability of the mean logits for the naive ensemble, and use the probability score proposed in Section 3.3 for our Split-Ensemble. A single threshold is used to detect OOD with score lower than the threshold.

B. Pseudo Code of Split-Ensemble Training

The Pseudo code of Split-Ensemble training is available in Algorithm 1.

C. Additional Results and Visualizations

In this section, we provide additional results in comparison with baseline methods in different settings as well as ablation results on our design choices following the discussion in Section 5.3.

Subtask grouping strategy In Section 3.1, we propose to use the group of classes that are semantically-close to form each subtasks of the complementary task splitting. We illustrate this concept in Figure 4, showing the natural semantic similarity we utilized to group our subtasks. Furthermore, we verify this intuition against have random assignment of classes to each subtask. As illustrated in Table 7, having semantically-close subtask grouping significantly improves the OOD detection ability of the Split-Ensemble model over that of random grouping. This improvement is more significant with more subtask splittings. We believe that semantic grouping of subtasks help the submodels to better learn the difference between ID classes and OOD classes of the subtask, as the semantically-close ID classes may share more distinct features comparing to other classes.

Algorithm 1 Training the Split-Ensemble model

```
# Initialization and preparation
Load dataset {X, Y}
Subtask label conversion Y → Ŷ_i as Equation (1)
Initialize Split-Ensemble F with all submodels f_i sharing backbone model
# Split ensemble training
while Training do
    Update F to minimize L_ens in Equation (4) with SGD
    # Iterative splitting and pruning
    if Epoch % Prune_Interval == 0 then
        # Splitting
        if ∃ branch in F with multiple submodel f_i sharing all layers then
            for Layers in the branch shared only by f_i do
                Compute sensitivity map following Equation (5) for each f_i
                Compute MCT of the layer following Equation (6)
                if MCT < threshold then
                    Split branch at the layer
                    Break
                end if
            end for
        end if
        # Pruning
        if FLOPs > target then
            for All submodels f_i do
                Compute I^i_S for all S following Equation (7)
                Rank I^i_S to decide prunable structures with min_S I^L_S
                Remove structures prunable for (all) corresponding submodels
            end for
        end if
    end if
end while
```

Table 7. **Ablation on subtask grouping strategy.** Models are trained on CIFAR-100. OOD detection is against the CIFAR-10 dataset.

# splits	Subtask grouping	Accuracy	AUROC
2	Random	77.3	78.9
	Semantic	**77.8**	**79.6**
4	Random	77.3	77.5
	Semantic	**77.5**	**79.1**
5	Random	77.4	77.3
	Semantic	**77.9**	**78.9**

Number of subtask splittings We conducted an analysis to explore the impact of the number of splits on the accuracy and OOD detection performance of the Split-Ensemble model. Unlike traditional ensemble that repeatedly learn the same task with more submodels, Split-Ensemble always learns a complementary subtask splitting corresponding to the original task. Increasing the amount of splits will therefore enable each submodel to learn a simpler subtasks with less ID classes, intuitively leading to a model architecture with more yet smaller branches. As shown in Table 8, the Split-Ensemble accuracy is not sensitive to the number of splits, showing the scalability of our learning algorithm. For OOD detection, a larger number of splits enables each submodel to learn its OOD-aware objective more easily, therefore leading to better AUROC. Yet the performance may suffer from aggressive pruning with too much branches in the Split-Ensemble, as observed with a large MCT threshold in Table 6. An interesting future direction would be automatically design the amount of subtask splitting and the grouping of each subtask during the training process to better fit the subtasks to the Split-Ensemble architecture.

Additional OOD detection results on CIFAR10 with SC-OOD benchmark We further compare our methods with previous state-of-the-art methods. In Table. 9, our Split-Ensemble model outperforms single-model approaches in OOD detection without incurring additional computational costs or requiring extra training data. Its consistent high performance across key metrics highlights its robustness and efficiency, underscoring its practical utility in OOD tasks. In Table. 10,

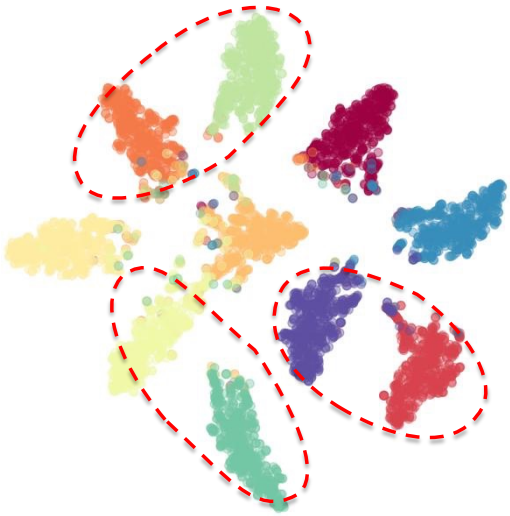

Figure 4. **TSNE visualization of learned features from ResNet-18 with CIFAR10 test set.** Samples with different class labels are marked in different colors. The adjacent features within the dashed lines are to be grouped into one subtask.

Table 8. **Ablation on number of splits.** Models are trained on CIFAR-100. OOD detection is against the CIFAR-10 dataset. All models are constrained with single-model computation cost.

# splits	2	4	5	8	10
Accuracy	77.7	**78.0**	77.9	77.5	77.3
AUROC	78.1	78.2	79.9	**80.4**	77.3

our Split-Ensemble model consistently outshines other ensemble-based methods in both image classification and OOD detection, achieving top rankings across all key metrics, which underscores the model's efficiency and effectiveness.

Robustness to corruption We evaluated our method and ensemble baselines on the corrupted dataset, CIFAR-10-C. In Table 11, We follow Ovadia et al. (2019) to report averaged top-1 accuracy (%) and ECE metric (%) across different corruption types for the fixed corruption severity level, which has a 1 to 5 range. It can be seen that even under data corruptions, our method has either the best or the second best in accuracy while being consistently well-calibrated and low-complexity among the ensemble baselines.

Model activation map visualization We visualize the learned feature map activations of a Split-Ensemble model across different layers using Score-CAM (Wang et al., 2020) in Figure 5. The shared feature maps, delineated by dashed lines, represent the common features extracted across different submodels, emphasizing the model's capacity to identify and leverage shared representations. The distinct feature maps outside the dashed boundaries correspond to specialized features pertinent to individual sub-tasks, demonstrating the Split-Ensemble model's ability to focus on unique aspects of the data when necessary. This visualization underscores the effectiveness of the Split-Ensemble architecture, highlighting its dual strength in capturing both shared and task-specific features within a single, cohesive framework, thereby bolstering its robustness and adaptability in handling diverse image classification and OOD detection tasks.

Model architecture visualization We visualize the Split-Ensemble models achieved under different MCT thresholds in Figure 6, as discussed in Table 6 of Section 5.3. Models here use 5 splits and are trained on CIFAR-100 dataset with ResNet-18 as backbone. With a larger MCT threshold, the model will split into more branches at earlier layers. Meanwhile the model will also be pruned more aggressively to keep the overall computation cost unchanged. We can clearly see that with a proper MCT threshold, our method can learn a tree-like Split-Ensemble architecture with different submodels branching out at different layers, as designed by our iterative splitting and pruning algorithm in Section 4.